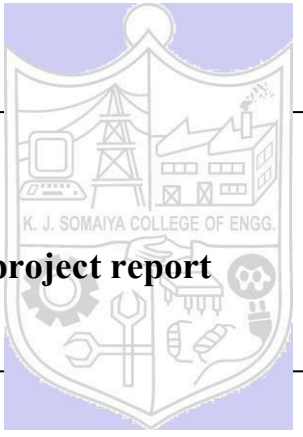
SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

KJSSE/AI&DS/SYBTECH /SEM-IV/AI/2025-26



Title: AI Mini project report

Title: AI Maze solver game using BFS and A* methods

Batch: B-1 **Roll No:16014224081**

Group: Mayank Minni – 16014224072**,** Bijeet Nayek–16014224081**,** Divy Pachauri - 16014224083

Aim:

To design and develop an interactive visual maze solver that demonstrates and compares two AI-based pathfinding algorithms **Breadth-First Search (BFS)** and **A* (A-Star)** on a grid-based environment, enabling users to observe how each algorithm explores paths and finds the shortest route from a start point to an end point in real time.

Resources needed:

Software: Python 3.x, Pygame library, Visual Studio Code.
Hardware: Laptop/Desktop PC.

Theory:

1. State Space Representation: In AI, every problem can be thought of as a space of possible states and the moves between them. In this project, each cell in the grid is a State, moving from one cell to a neighbouring cell is a Transition, the green cell is the Initial State, and the red cell is the Goal State. Solving the maze means finding a sequence of transitions that leads from the start state to the goal state. This is a classic Graph Search Problem.

2. BFS Algorithm (Breadth-First Search): BFS is an uninformed or blind search algorithm, meaning it has no idea where the goal is. It explores all cells that are 1 step away first, then all cells that are 2 steps away, and so on — just like a wave spreading outward equally in all directions. It uses a Queue (FIFO) data structure, where cells are added to the back and removed from the front. BFS guarantees the shortest path in terms of number of steps because it always explores closer cells before farther ones.

3. A Algorithm (A-Star Search):* A* is an informed or smart search algorithm. Instead of exploring blindly, it uses a cost formula to decide which cell to explore next: f(n) = g(n) + h(n), where g(n) is the number of steps already taken from the start, and h(n) is a heuristic estimate of how far the current cell is from the goal. It uses a Priority Queue (min-heap), always picking the cell with the lowest f(n) score. This guides A* directly toward the goal, making it much faster and more efficient than BFS.

4. Heuristic Function — Manhattan Distance: A heuristic is an educated guess used to guide the search. Since movement in this maze is only allowed in 4 directions (up, down, left, right), this project uses the Manhattan Distance as h(n). It is calculated as the sum of the horizontal and vertical distance between two cells: h(n) = |row1 − row2| + |col1 − col2|. This heuristic never

overestimates the actual distance, which makes it admissible and ensures A* always finds the optimal shortest path.

Implementation / Code

The project is built as a single Python file using Pygame. It has the following main parts:

Grid Structure: A 25×25 grid where each cell can be open (white), a wall (gray), the start (green), or the end (red). The grid is stored as a 2D list where 0 means open and 1 means wall.

python
```
grid = [[0] * COLS for _ in range(ROWS)]
start = None
end = None
```

BFS Function:

python
```
def bfs():
    queue = deque([(start, [])])
    visited = set()
    while queue:
        node, p = queue.popleft()
        if node in visited:
            continue
        visited.add(node)
        if node == end:
            path = p + [node]
            return
        for n in neighbors(node):
            queue.append((n, p + [node]))
```


A Function:*

python
```
def astar():
    pq = [(0, start, [])]
    visited = set()
    while pq:
        cost, node, p = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == end:
            path = p + [node]
            return
        for n in neighbors(node):
            heapq.heappush(pq,
            (len(p) + heuristic(n, end), n, p + [node]))
```

Full Code :

```
import pygame
import sys
from collections import deque
import heapq

#Settings
ROWS, COLS = 25, 25
UI_BAR = 52

pygame.init()
info = pygame.display.Info()

max_w = int(info.current_w * 0.9)
max_h = int(info.current_h * 0.9) - UI_BAR

CELL   = min(max_w // COLS, max_h // ROWS)
GRID_W = CELL * COLS
GRID_H = CELL * ROWS
WIDTH  = GRID_W
HEIGHT = GRID_H + UI_BAR

screen = pygame.display.set_mode((WIDTH, HEIGHT), pygame.RESIZABLE)
pygame.display.set_caption("AI Maze Solver (BFS vs A*)")

#Colors
WHITE  = (255, 255, 255)
BLACK  = (30,  30,  30)
GREEN  = (0,   200, 100)
RED    = (255, 80,  80)
BLUE   = (50,  120, 255)
YELLOW = (255, 215, 0)
CYAN   = (0,   255, 255)
GRAY   = (60,  60,  60)
DIM    = (160, 160, 160)

# state
grid    = [[0] * COLS for _ in range(ROWS)]
start   = None
end     = None
visited = set()
path    = []
mode    = "BFS"

# recalc
def recalc(w, h):
    global CELL, GRID_W, GRID_H, WIDTH, HEIGHT
    avail_w = w
    avail_h = h - UI_BAR
    CELL   = max(4, min(avail_w // COLS, avail_h // ROWS))
    GRID_W = CELL * COLS
    GRID_H = CELL * ROWS
    WIDTH  = GRID_W
    HEIGHT = GRID_H + UI_BAR

def ui_font(target_w):
    """Return the largest font size where both hint lines fit within target_w."""
    line1 = f"Mode: {mode}  |  S: Start   E: End   Click: Wall   RClick: Erase   1/2:
BFS/A*"
    line2 = "SPACE: Solve   C: Clear"
    for size in range(17, 7, -1):
```

```
        f = pygame.font.SysFont("consolas", size)
        if f.size(line1)[0] <= target_w - 16 and f.size(line2)[0] <= target_w - 16:
            return f, line1, line2
    return pygame.font.SysFont("consolas", 8), line1, line2

#draw
def draw():
    screen.fill(BLACK)

    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] == 1:
                color = GRAY
            elif (r, c) in path:
                color = YELLOW
            elif (r, c) in visited:
                color = BLUE
            else:
                color = WHITE
            pygame.draw.rect(screen, color,
                             (c * CELL, r * CELL, CELL - 1, CELL - 1))

    if start:
        pygame.draw.rect(screen, GREEN,
                         (start[1] * CELL, start[0] * CELL, CELL, CELL))
    if end:
        pygame.draw.rect(screen, RED,
                         (end[1] * CELL, end[0] * CELL, CELL, CELL))

    # UI bar - two lines, auto-sized
    bar_y = GRID_H
    pygame.draw.rect(screen, (20, 20, 20), (0, bar_y, WIDTH, UI_BAR))

    f, line1, line2 = ui_font(WIDTH)
    lh    = f.get_height()
    gap   = 3
    total = lh * 2 + gap
    y0    = bar_y + (UI_BAR - total) // 2

    screen.blit(f.render(line1, True, CYAN), (8, y0))
    screen.blit(f.render(line2, True, DIM),  (8, y0 + lh + gap))

    pygame.display.update()

#helpers
def cell_at(mx, my):
    r, c = my // CELL, mx // CELL
    if 0 <= r < ROWS and 0 <= c < COLS:
        return r, c
    return None

def neighbors(node):
    r, c = node
    result = []
    for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc < COLS and grid[nr][nc] == 0:
            result.append((nr, nc))
    return result


def bfs():
```

```
    global visited, path
    queue   = deque([(start, [])])
    visited = set()
    while queue:
        node, p = queue.popleft()
        if node in visited:
            continue
        visited.add(node)
        if node == end:
            path = p + [node]
            return
        for n in neighbors(node):
            queue.append((n, p + [node]))
        draw()
        pygame.time.delay(15)
        for e in pygame.event.get():
            if e.type == pygame.QUIT:
                pygame.quit(); sys.exit()


def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar():
    global visited, path
    pq      = [(0, start, [])]
    visited = set()
    while pq:
        cost, node, p = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == end:
            path = p + [node]
            return
        for n in neighbors(node):
            heapq.heappush(pq, (len(p) + heuristic(n, end), n, p + [node]))
        draw()
        pygame.time.delay(15)
        for e in pygame.event.get():
            if e.type == pygame.QUIT:
                pygame.quit(); sys.exit()


recalc(WIDTH, HEIGHT)

while True:
    for event in pygame.event.get():

        if event.type == pygame.QUIT:
            pygame.quit(); sys.exit()

        if event.type == pygame.VIDEORESIZE:
            recalc(event.w, event.h)
            screen = pygame.display.set_mode((WIDTH, HEIGHT), pygame.RESIZABLE)

        if pygame.mouse.get_pressed()[0]:
            mx, my = pygame.mouse.get_pos()
            cell = cell_at(mx, my)
            if cell and cell != start and cell != end:
                grid[cell[0]][cell[1]] = 1
```

```
        if pygame.mouse.get_pressed()[2]:
            mx, my = pygame.mouse.get_pos()
            cell = cell_at(mx, my)
            if cell:
                grid[cell[0]][cell[1]] = 0

        if event.type == pygame.KEYDOWN:
            mx, my = pygame.mouse.get_pos()

            if event.key == pygame.K_s:
                cell = cell_at(mx, my)
                if cell:
                    start = cell
                    grid[cell[0]][cell[1]] = 0

            if event.key == pygame.K_e:
                cell = cell_at(mx, my)
                if cell:
                    end = cell
                    grid[cell[0]][cell[1]] = 0

            if event.key == pygame.K_c:
                grid    = [[0] * COLS for _ in range(ROWS)]
                start   = None
                end     = None
                visited = set()
                path    = []

            if event.key == pygame.K_1:
                mode = "BFS"
            if event.key == pygame.K_2:
                mode = "A*"

            if event.key == pygame.K_SPACE and start and end:
                visited = set()
                path    = []
                if mode == "BFS":
                    bfs()
                else:
                    astar()

    draw()
```

Results: (Softcopy submission of Summary Document)

- Shortest Path (A*): A* explores far fewer cells and reaches the goal faster by always moving in the most promising direction toward the end point.

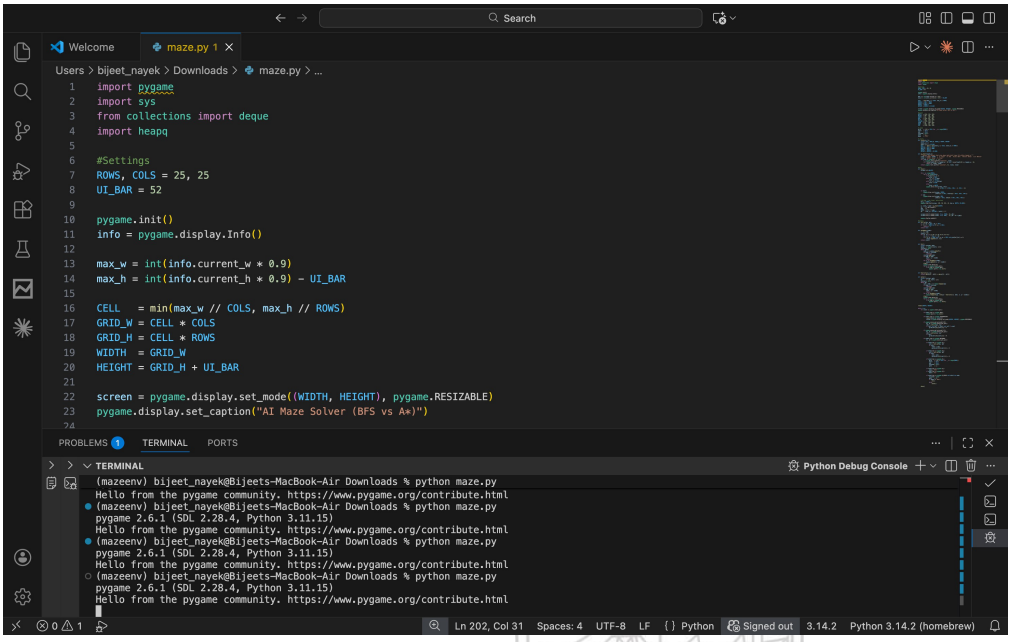
- BFS Path: BFS spreads out in all directions equally, exploring many more cells before finding the goal, but still guarantees the shortest path.

- Comparison: When both algorithms are run on the same maze, A* visually explores a much smaller area (fewer blue cells) compared to BFS, yet both arrive at the same shortest yellow path.

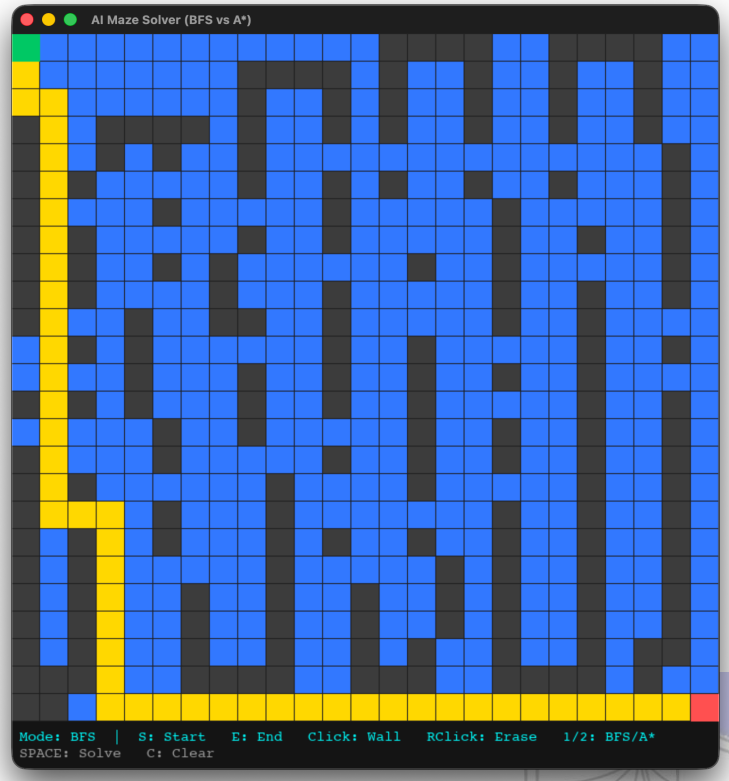
- Visual Output: Explored cells appear in blue, the final path appears in yellow, start in green and end in red.

Working Snapshots of the project:

1. Running the code



2. Using BFS mode



3. Using A* algorithm

Outcome:

- Understood the clear difference between uninformed search (BFS) and informed search (A*) and how a heuristic makes the algorithm significantly smarter and faster.
- Practically implemented both graph traversal algorithms from scratch in Python, connecting AI theory directly to a working visual application.
- Visualized core AI concepts like state space, nodes, transitions, visited states, and shortest path on an interactive grid, making them very easy to understand and explain.

Outcome related to the course :

Formulate the problem as a state space representation.

Conclusion:

This mini project successfully shows that smart AI algorithms like A* are far more efficient than blind algorithms like BFS when it comes to finding paths. BFS explores every possible direction equally and wastes time visiting cells that are nowhere near the goal. A*, guided by the Manhattan Distance heuristic, always moves toward the goal intelligently and finds the same shortest path while exploring far fewer cells. By building this as a live animated visual tool using Python and Pygame, the project makes fundamental AI concepts like state space search, heuristics, and optimal pathfinding very simple and fun to understand.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date